

Data Vortex 2026 - Phase 2 Logic Explanation

1. Selected questions

This submission solves one question from each difficulty level:

Difficulty	Selected question
Easy	E3 - Average Engagement by Platform
Medium	M2 - Do High Follower Users Get More Engagement?
Hard	H3 - Platform Performance Compared With Its Own Average

The queries are executed against SQLite tables created from the cleaned users and posts CSV files.

2. Database structure

The analysis uses two tables.

Users table

The `users` table contains:

- `user_id`
- `location`
- `language`
- `account_created`
- `follower_count`

The `user_id` column uniquely identifies each user.

Posts table

The `posts` table contains:

- `post_id`
- `user_id`
- `platform`
- `text_content`
- `timestamp`
- `likes`
- `shares`
- `comments`
- `timestamp_original`

The `post_id` column uniquely identifies each post.

The two tables are connected through:

```
users.user_id = posts.user_id
```

This relationship allows post-level engagement to be analyzed using user-level follower information.

3. E3 - Average Engagement by Platform

Objective

The E3 question asks us to calculate the average likes, shares, and comments for each platform and identify the platform with the highest average total engagement.

Query logic

The query uses a Common Table Expression called `platform_engagement`.

```
WITH platform_engagement AS (
```

The CTE first groups posts by platform:

```
GROUP BY platform
```

For each platform, it calculates:

```
AVG(likes)
AVG(shares)
AVG(comments)
```

These expressions calculate the average individual engagement measures.

The query also calculates total engagement:

```
likes + shares + comments
```

Because some posts have missing likes, the query uses:

```
COALESCE(likes, 0) + shares + comments
```

`COALESCE(likes, 0)` temporarily treats a missing like value as zero for the total-engagement calculation. The cleaned source dataset is not changed.

Posts with missing platform values are excluded:

```
WHERE platform IS NOT NULL
```

This is necessary because a post without a platform cannot be assigned to a named platform.

Finally, the results are sorted from highest to lowest average total engagement:

```
ORDER BY average_total_engagement DESC
```

Therefore, the first result row identifies the platform with the highest average total engagement.

E3 result

The results show that **Reddit** has the highest average total engagement, with an average of approximately **3,550.43** interactions per post.

4. M2 - Do High Follower Users Get More Engagement?

Objective

The M2 question divides users into two groups:

- High follower users: at least 25,000 followers.
- Low follower users: fewer than 25,000 followers.

The groups are then compared using average engagement per post.

Query logic

The first CTE calculates engagement for each post:

```
COALESCE(likes, 0) + shares + comments  
AS total_engagement
```

The second CTE classifies users with a `CASE` expression:

```
CASE  
WHEN follower_count >= 25000
```

```
    THEN 'High follower (>=25000)'  
    ELSE 'Low follower (<25000)'  
END
```

This directly implements the thresholds given in the questionnaire.

The user groups are then joined to the post engagement data:

```
LEFT JOIN post_engagement  
  ON user_groups.user_id = post_engagement.user_id
```

A left join preserves the user-group records even if a user has no matching post.

The query groups the joined records by follower group:

```
GROUP BY user_groups.follower_group
```

It calculates:

- Number of posts.
- Number of distinct users.
- Average engagement per post.

The final result is ordered by average engagement per post.

M2 result

The results are:

Follower group	Average engagement per post
Low follower users	3,535.84
High follower users	3,508.45

In this dataset, the low-follower group has a slightly higher average engagement per post than the high-follower group.

This result describes the observed dataset. It does not prove that having fewer followers causes higher engagement.

5. H3 - Platform Performance Compared With Its Own Average

Objective

The H3 question asks us to identify posts whose engagement is at least twice the average engagement of their own platform.

Query logic

The query uses two Common Table Expressions.

First CTE: post engagement

The first CTE calculates total engagement for each post:

```
COALESCE(likes, 0) + shares + comments  
AS total_engagement
```

Posts without a platform are excluded:

```
WHERE platform IS NOT NULL
```

Second CTE: platform averages

The second CTE calculates the average total engagement separately for every platform:

```
AVG(total_engagement)  
AS platform_average_engagement
```

The data is grouped by platform:

```
GROUP BY platform
```

This is important because each post is compared with the average of its own platform, not with one global average.

Comparing each post with its platform average

The query joins each post to its platform average:

```
INNER JOIN platform_averages  
ON post_engagement.platform = platform_averages.platform
```

It then applies the questionnaire threshold:

```
WHERE post_engagement.total_engagement  
      >= 2 * platform_averages.platform_average_engagement
```

This keeps only posts whose engagement is at least two times their own platform average.

The result is ordered by platform and descending total engagement:

```
ORDER BY  
  post_engagement.platform,  
  post_engagement.total_engagement DESC
```

H3 result

The query identified **139 exceptional posts** that reached at least twice the average engagement of their own platform.

This is a platform-relative anomaly analysis. It does not simply select posts from the platform with the highest overall average.

6. Data-handling assumptions

The following assumptions were applied consistently:

1. Missing likes are treated as zero only when calculating total engagement.
2. Individual average-likes calculations use SQL `AVG`, which ignores missing likes.
3. Posts with missing platforms are excluded from platform-specific comparisons.
4. No missing text, platform, likes, or other values were fabricated.
5. The original cleaned CSV files remain unchanged.
6. All results are generated dynamically from SQLite tables and SQL queries.

7. Reproducibility

The database and query outputs were generated with:

```
python "phase2\run_selected_queries.py"
```

The query outputs were then converted into JPEG screenshots using:

```
python "phase2\create_screenshots.py"
```

The SQL files used for this explanation are:

```
phase2\sql\E3_average_engagement_by_platform.sql  
phase2\sql\M2_follower_groups.sql  
phase2\sql\H3_platform_anomalies.sql
```

The generated result files are stored in:

```
phase2\results\
```

The output screenshots are stored in:

```
phase2\screenshots\
```